LAUNDRY DAY NYC

Laundry Day NYC — System Review & Ideas Menu

Codebase review & a numbered menu of ideas to pick from

A full read-through of the codebase, plus a menu of ideas to pick from. Each idea is numbered so you can just reply with the numbers you want. Tags: **[Impact]** how much it helps · **[Effort]** rough build cost · **[New dep]** if it needs a new service.

---

1. How the system works today

Stack: Next.js 14 app on Vercel · Google Sheets as the database · Resend for email · Cloudinary for photos · Vercel Cron for scheduling. Cost is near-$0, which is great.

Two service areas: Uptown (Fri pickup / Sat pickup + returns) and Downtown (Tue pickup / Thu pickup + returns). "Day 2" is a combined route: Thursday/Saturday returns last week's clean laundry *and* picks up new bags.

Data lives in Google Sheet tabs: customer lists per area, a Keys tab (building access), and app-managed tabs the code auto-creates — Pickup Responses, Route Edits, Route Order, Driver Progress, Driver Issues, Dropoff Photos, Email Bounces, Opt-outs, Settings.

Reminder flow: - A cron fires at 7:20 AM ET. Tuesday → downtown "main" email to everyone; Thursday → downtown "remaining" (unconfirmed) + "confirmed today"; Friday → uptown main; Saturday → uptown remaining + confirmed. Monday → no emails, just a stale-customer report to you. - Customer taps the button → `/pickup` → picks a day → it's logged for the week. - The dashboard *also* has a parallel manual flow (copy BCC list, copy subject, copy body, paste into Gmail). This predates the cron and now overlaps with it.

Route building: A hand-coded heuristic sorts stops geographically — regex that classifies each address as east/west side and estimates a cross-street using the Manhattan Address Algorithm, plus hardcoded special cases (953 Columbus always last, a couple of standing pickups, permanent cycle stops). The driver can then drag/reorder, and that order is saved. ETAs are estimated from one calibrated historical day (May 23).

Driver app: PIN login → a polished green "current stop" card (address, access method, big Mark Collected, Directions, Can't enter, No bag) plus an "All stops" list with drag-reorder and ETAs. Photo capture for issues and drop-off proof. This part is genuinely well designed.

Admin dashboard: PIN login → one long scrolling page: four action cards on top, then results render inline, followed by permanent Live Tracking, Driver Statistics, Email Bounces, and Settings sections all stacked.

---

2. What's already strong

- The driver app is clean, mobile-first, and thoughtful (companion/multi-stop callouts, photo proof, offline-ish optimistic updates, test mode).
- Robust email plumbing — retries, batching, bounce/complaint webhook, auto-unsubscribe, opt-outs, admin failure alerts, same-day dedup.
- Live driver tracking with auto-refresh and issue photos is a real operational asset.
- Near-zero infra cost.

---

3. Pain points I found

- **Admin is one long jumble.** No navigation. Daily "do this now" tools sit in the same scroll as analytics and settings. The color theme (purple/teal/orange gradients) clashes with the driver app's clean green brand — the two products don't look related.
- **Two reminder systems coexist.** The automated cron and the manual copy-paste BCC flow do the same job. That's the "complicated / jumbled" feeling — it's unclear which one is the source of truth.

- **Route logic is brittle.** It's regex over street names with no real geocoding, distances, or traffic. New/unusual addresses fall back to crude guesses. The same ~60 lines of sort logic are duplicated in the server *and* the dashboard client — easy to drift out of sync.
- **No real map.** Drivers get a list and a per-stop "Directions" link, but no single optimized multi-stop map view.
- **Customers are edited by hand in Google Sheets.** No admin UI to add/search/edit a customer, set their default day, or pause them.
- **Styling is all inline objects.** A redesign means touching thousands of inline style lines unless you move to a design-token/component system first.

---

4. Ideas — Reminders & customer flow

4.1 — Kill the manual email flow, make "send" one button. Impact: High Effort: Low Retire the copy-BCC / copy-subject / copy-body cards. Replace with a single "Send reminders now" button and a clear status line ("Auto-send is ON. Next: Friday 7:20 AM"). One source of truth.

4.2 — Add SMS/text reminders. Impact: Very High Effort: Med New dep: Twilio Texts get read and answered far more than email for time-sensitive pickups. Customer replies "FRI" / "SAT" to confirm. This is the single biggest lever on confirmation rates.

4.3 — One-tap confirm in the message. Impact: High Effort: Low You already have per-day deep links. Make the email/text buttons confirm in one tap with zero typing (today the page still asks for their email).

4.4 — "You're on unless you reply STOP" for regulars. Impact: High Effort: Med Customers who pick up most weeks get auto-added to the route and only need to opt *out* this week. Cuts reminder volume and no-replies. You already track per-customer history for stale detection — same data feeds this.

4.5 — Customer preference / pause page. Impact: Med Effort: Med A self-serve page to set a default day, pause for vacation, or change frequency — fewer one-off emails to you.

4.6 — Live "driver is ~20 min away" customer alert. Impact: Very High / differentiator Effort: Med You already compute per-stop ETAs in the driver app. Text each customer a tightening window as the driver approaches so bags actually make it out by pickup. Almost no competitor at this scale does this.

4.7 — Auto drop-off proof to the customer. Impact: Med Effort: Low You already capture a drop-off photo. Optionally text/email it to the customer as "delivered" proof.

4.8 — Weather-aware nudge. Impact: Low-Med Effort: Low "Rain tomorrow — double-bag or leave inside the vestibule."

---

5. Ideas — Route optimization

5.1 — Real geocoding + route optimization API. Impact: Very High Effort: Med-High New dep: Google Routes / Mapbox Optimization / open-source OSRM Replace the regex heuristic with actual lat/lng and a solver that returns the true shortest order with live traffic. Geocode each address once and cache it on the Keys/customer row.

5.2 — Learn real times from your own history. Impact: High Effort: Med You already log per-stop timestamps and durations (Driver Stats). Use that to calibrate stop-to-stop and per-building service times instead of one hardcoded May 23 sample — the estimates and the optimizer both get sharper every week.

5.3 — Full multi-stop map view for the driver. Impact: High Effort: Med One map with all stops numbered in route order and tap-to-navigate, instead of per-stop links. Big quality-of-life win on the road.

5.4 — "Re-optimize" button with manual override kept. Impact: Med Effort: Low once 5.1 exists Driver/admin taps to re-solve (e.g., after late signups), but any manual drag still wins — which is already your model.

5.5 — De-duplicate the sort logic. Impact: Med (maintainability) Effort: Low The client copy in the dashboard and the server copy in `sheets.js` should be one shared module so they can't drift.

5.6 — Parking / one-way / building-cluster awareness. Impact: Med Effort: Med Encode known parking spots and group same-building stops automatically (the driver app already has a "companions" concept to build on).

---

6. Ideas — Admin redesign (the main ask)

6.1 — Add real navigation (tabs or sidebar). Impact: Very High Effort: Med Split the one-scroll into clear sections: **Today** · **Route** · **Customers** · **Analytics** · **Settings**. This alone fixes most of the "jumbled / confusing" feeling.

6.2 — A day-aware "Today" home. Impact: Very High Effort: Med The landing view reads the day of week and shows only what matters now — e.g., Friday morning surfaces uptown confirmation count, live tracking, and a single send button. No hunting.

6.3 — Unify the brand with the driver app's green. Impact: High Effort: Low-Med Adopt the driver palette (the clean `PALETTE` greens) across admin so the two feel like one product. Drop the purple/orange gradients.

6.4 — Move to a design system before restyling. Impact: High (foundation) Effort: Med Introduce Tailwind or shared style tokens/components so future visual changes are one edit, not thousands of inline objects. Worth doing first if you want repeated polish.

6.5 — Live tracking becomes the hero on pickup days. Impact: High Effort: Med Map + progress + issues front-and-center, auto-refreshing, instead of a section you scroll to and manually "Load."

6.6 — Real customer management UI. Impact: High Effort: Med-High Search/add/edit customers, set default day, opt-out, view pickup history — instead of hand-editing the Google Sheet.

6.7 — Make admin fully mobile-friendly. Impact: Med Effort: Low-Med You'll often check status from your phone; the current dense tables don't love small screens.

6.8 — Optional dark mode. Impact: Low Effort: Low

---

7. Outside-the-box / bigger bets

7.1 — One app, role-based views. Impact: High Effort: High Merge admin + driver into a single codebase with shared design system and role switching, instead of two separate pages with duplicated logic.

7.2 — Payments / billing. Impact: High Effort: Med-High New dep: Stripe — already connected here Capture bag weight at pickup and bill automatically; subscriptions for weekly regulars.

7.3 — WhatsApp channel. Impact: Med-High Effort: Med Many NYC customers prefer WhatsApp; rich confirm buttons and delivery photos land natively.

7.4 — Retention dashboard. Impact: Med Effort: Med You already detect stale customers — turn it into churn/retention/revenue trends and win-back nudges.

7.5 — Move off Google Sheets eventually. Impact: Med (reliability/scale) Effort: High Sheets is charming and free but slow and fragile as a DB. A lightweight Postgres (e.g., Vercel/Supabase free tier) would speed everything up and de-risk concurrent edits — worth keeping on the radar, not urgent.

---

8. Suggested first wave (my recommendation)

If you want the biggest visible improvement for the least risk, I'd pair: **6.1 + 6.2 + 6.3** (navigation, day-aware home, unified green brand) to fix the admin look-and-feel, **4.1** (kill the manual email flow) to de-clutter the workflow, and **5.5** (de-dupe route logic) as cheap cleanup. Then the high-impact bets — **4.2** / **4.6** (SMS + live ETA alerts) and **5.1** (real route optimization) — as the next round.

Tell me which numbers you like and I'll turn them into a concrete implementation plan for a Claude Code update.